

富途前端面试

富途前端面试

1. 有效括号匹配
2. 判断 b 是否是 a 的子集 (a 和 b 有重复元素, 要求 b 的同个元素出现次数 $\leq$ a 的同个元素出现次数)
3. 离职原因
4. 最有挑战的项目
5. 技术分享做过哪些
6. 如何发现用户痛点并解决
7. vue 和 react 的相同点和不同点
8. 单项数据流和双向数据绑定的原理, 区别
9. vue3 为什么用 proxy 改写数据劫持
10. 项目是否用到 typescript
11. 项目是否用到自动化测试
12. props 数据流向
13. ref 和 reactive 区别
14. 一个页面输入到展示过程
15. react 页面白屏检测
16. csrf 原理和解决方案
17. 302 怎么确定重定向路径
18. 移动端怎么适配
19. 性能优化
20. 全排列
21. js 对象数组转树形结构
22. `promise(A).catch(f1).then(f2)`, f1 执行后 f2 回执行吗, 为什么

参考答案

1. 有效括号匹配：

这是一个经典的算法问题，通常使用栈（Stack）数据结构来解决。

当遇到一个左括号时，将其压入栈中。

当遇到一个右括号时，检查栈顶元素是否为对应的左括号，如果是则弹出栈顶元素，否则括号不匹配。

2. 判断 b 是否是 a 的子集：

可以使用哈希表（HashMap）来记录 a 中每个元素的出现次数。

然后遍历 b 中的元素，对于每个元素，检查其在哈希表中的计数是否足够（即 b 中的元素出现次数不超过 a 中的出现次数）。

4. 最有挑战的项目：

这个问题需要结合个人经历来回答，可以描述一个自己在工作中遇到的最大挑战，以及如何克服这些挑战。

5. 技术分享做过哪些：

可以分享自己参与过的技术项目、技术选型、问题解决经验等。

也可以分享自己学习新技术的心得体会。

6. 如何发现用户痛点并解决：

通过用户调研、数据分析、用户反馈等方式发现用户痛点。

针对发现的痛点，制定解决方案，并进行迭代优化。

7.vue 和 react 的相同点和不同点：

相同点：都是现代前端框架，支持组件化开发，提高代码复用性和可维护性。

不同点：Vue 采用双向数据绑定，而 React 采用单向数据流；Vue 的模板语法更加简洁易懂，而 React 的 JSX 需要一定的学习成本；Vue 的社区和生态相对较小，但正在快速发展；React 的社区和生态则更加庞大和成熟。

8. 单项数据流和双向数据绑定的原理、区别：

单项数据流：数据只能从一个方向流动，即从父组件到子组件。这有助于保持组件的独立性和可预测性。

双向数据绑定：数据可以在父组件和子组件之间双向流动。这有助于简化用户输入等交互场景的实现，但也可能导致数据流变得复杂和难以预测。

9.vue3 为什么用 proxy 改写数据劫持：

Vue3 使用 Proxy 来改写数据劫持，主要是因为 Proxy 提供了更强大的数据劫持能力，可以监听对象属性的读取、写入、删除等操作。

这使得 Vue3 可以更好地处理复杂的数据结构和嵌套对象，提高数据响应式的性能和灵活性。

10. 项目是否用到 typescript:

这个问题需要结合个人或团队的项目实际情况来回答。

如果项目使用了 TypeScript，可以简单介绍一下 TypeScript 在项目中的作用和优势。

11. 项目是否用到自动化测试:

这个问题同样需要结合个人或团队的项目实际情况来回答。

如果项目使用了自动化测试，可以介绍一下自动化测试的工具、流程、优势以及遇到的挑战和解决方案。

12.props 数据流向:

Props 是 Vue 中用于父组件向子组件传递数据的一种机制。

Props 的数据流向是单向的，即只能从父组件传递到子组件，子组件不能修改 props 的值。

13.ref 和 reactive 区别:

ref 主要用于基本数据类型的响应式处理，而 reactive 则用于对象或数组的响应式处理。

ref 在模板中访问时需要通过 .value 属性来获取值，而 reactive 则不需要。

14. 一个页面输入到展示过程:

用户输入数据后，数据会被发送到服务器进行处理。

服务器处理完数据后，将结果返回给前端。

前端接收到数据后，将其展示在页面上。

15.react 页面白屏检测:

可以通过在页面中添加错误边界（Error Boundary）组件来捕获渲染过程中的错误，并显示备用 UI 来避免白屏。

也可以使用监控工具来检测页面加载状态和响应时间，及时发现并处理白屏问题。

16.csrf 原理和解决方案:

CSRF（跨站请求伪造）攻击的原理是攻击者诱导受害者在已登录状态下访问恶意网站，该网站会向受害者的受信任网站发送伪造请求。

常见的解决方案包括使用验证码、检查 Referer 头、使用双重认证等。

17.302 怎么确定重定向路径：

HTTP 302 状态码表示临时重定向。

客户端在接收到 302 状态码后，会从响应头中的 Location 字段获取重定向的目标 URL。

18. 移动端怎么适配：

可以使用响应式设计、流式布局、弹性盒子（Flexbox）和网格布局（Grid Layout）等技术来实现移动端的适配。

同时，也可以使用媒体查询（Media Queries）来针对不同的屏幕尺寸和分辨率进行样式调整。

19. 性能优化：

性能优化可以从多个方面入手，包括代码优化、图片优化、网络优化、数据库优化等。

具体的优化策略需要根据项目的实际情况和需求来制定。

20. 全排列：

全排列是指从给定的一组元素中生成所有可能的排列组合。

可以使用递归或迭代的方法来实现全排列。

21.js 对象数组转树形结构：

可以根据对象的某个属性（如 id 和 parentId）来构建树形结构。

通过遍历对象数组，将每个对象放入一个以 id 为键的 Map 中，然后根据 parentId 找到其父节点，并将其添加到父节点的 children 数组中。

22.promise(A).catch(f1).then(f2)，f1 执行后 f2 会执行吗，为什么：

是的，f1 执行后 f2 会执行。

这是因为 Promise 的 .catch() 方法会返回一个新的 Promise，这个新的 Promise 会继承原 Promise 的状态和结果。

当 .catch() 捕获到错误并执行 f1 后，新的 Promise 的状态会变为 fulfilled（已完成），因此后续的 .then() 方法会接收到 fulfilled 状态并执行 f2。